SmartBrew AI - Group 28 - Lab 3 AI appendix

Updated 2026-10-21. This appendix records development assistance, the main supplied prompt and key interactions. The following record identifies the development and runtime models separately.

Record scope and prompt strategy

This appendix includes the main supplied request, key responses and clarifications. Markdown whitespace is normalized. The later Lab 1/course-policy context is explicitly summarized rather than represented as a full verbatim export. Generated source files in the Git repository are the code responses. This record contains no private model reasoning, invented prompts or student-written personal reflection.

Main Lab 2 Correction / Lab 3 Prompt

Student message from 20 October 2026. The assignment text below is the input supplied by the student, rather than a claim that the assistant authored it. Source attachments: feedback `G28Lab2 (1).pdf`, submitted `Lab2_G28.zip`, and the supplied Lecture 2 and Lecture 3 PDFs.

G28Lab2 (1).pdf is the feedback we got. zip file contains our lab2 submission. according to feedback, and lab lectures 2 and 3, and if you need, lab2s guidelines: "Lab assignment 2:

This lab is about requirements and planning. For task A, we will continue to use the tool Astah. For task B, we recommend word or another textual tool with a PDF capability.

Task 2a:

Create a UML use case diagram for the system as described in the [System description for all lab assignments](https://canvas.mdu.se/courses/20908/pages/system-description-for-all-lab-assignments "System description for all lab assignments") , including the feature of your own design as you added during lab 1.

To be marked complete, the diagram must cover the functionality of the system and contain the following:

- At least 3 Actors
- At least 8 Use cases with:
- For each use case, a textual description (right click the use case and select "open use case description"). You may use [this template](https://canvas.mdu.se/courses/20908/files/3005789?wrap=1)[Download this template](https://canvas.mdu.se/courses/20908/files/3005789/download?download_frd=1) for describing the use cases. Import it into Astah via Tool -> Template Settings -> UseCase Description -> Import and then Apply.
- Correct relationships between actors, between use cases, and between actors and use cases.
- A clear indication of the feature you added yourself (if any), for example by adding a note to the diagram.

Task 2b:

In this assignment, you will write user stories and create backlogs for the development.

Follow the following steps to complete this task:

1. Read Chapters 2 (on product backlogs), 3 (on sprint planning), and 6 (on sprint backlogs) of "Scrum and XP From the Trenches".
2. Based on the system description and your added feature, write down a list of at least 10 **user stories** for the SmartBrew system. Your user stories need to cover the complete functionality of the system according to the description. A typical user story has the form *"As a <role> I want <goal/desire> so that <benefit>";* you can refer to the template from the slides.
3. (starting from the user stories) Define a set of **requirements** covering the key functionalities of the system (the requirements should be expressed using the template from the slides)
4. Create an initial **product backlog** with the requirements (the initial product backlog contains at least ID, name, importance, estimate, description, how to demo) using the template provided in the slides.". we need to fix, and then do lab3:

Lab Assignment 3: Software Engineering with LLMs

The goal of this assignment is to engage with core software engineering tasks using Large Language Models (LLMs) as virtual assistants. You will observe, guide, and critically evaluate the software development lifecycle when augmented by AI.

You are required to complete **three main tasks**:

1. **Task A:** LLM-Assisted Requirements Elicitation
2. **Task B:** Code Generation & Implementation
3. **Task C:** Documentation & Repository Management

Choice of LLM

You are free to use **any LLM or AI assistant of your choice** (e.g., ChatGPT/GPT-4, Claude, Gemini, GitHub Copilot, Llama, etc.). However, you must explicitly document which model(s) and version(s) you used for each task.

Task A: Requirements Elicitation & Critical Comparison

1. **Elicitation with LLMs:**

Conduct a requirements elicitation process for the System Description used across all Labs using your chosen LLM. Generate the following three artifacts (matching the required format from Assignment 2):
- Functional and Non-Functional Requirements
- User Stories
- Product Backlog
2. **Prompting & Transcript:**

You must document the prompt strategy you used to generate these artifacts. Include the main prompts and key responses as an appendix or separate document.
3. **Comparison with Assignment 2:**

Compare the LLM-generated artifacts with the human-elicited version you submitted for Assignment 2.
- Highlight differences clearly (e.g., via side-by-side text, Git diff format with red/green highlights, or detailed annotated lists).
- Detail what the LLM improved, missing aspects it failed to identify, or unneeded/incorrect requirements it suggested.
- If you reject LLM suggestions, you **must provide a short justification** for each rejection.

(Note for students who did not submit Assignment 2: You may either complete Assignment 2 first and then perform the comparison, or perform the LLM elicitation directly while fully documenting and justifying every step of your interaction with the LLM).

Task B: Code Generation & Execution

1. **Functionality Selection:** Select a core set of **10 features/functionalities (not trivial)** from the requirements elicited in Task A.
2. **Implementation:** Realize/code these 10 functionalities with LLM assistance.
3. **Executable Prototype:** Ensure a working user experience. The system must run smoothly and demonstrate the selected 10 functionalities clearly. It is not required a complete portability of the project (Can work on only one specific OS, like macOS, linux and windows).
4. In case of complete code generation, you need to add the conversation. If you used the LLM just as an assistant to revise the code it is not required to include the conversation.

Task C: Repository & Documentation

1. **Git Repository:** Host your codebase in a Git repository (e.g., GitHub).
2. **Code Documentation & Comments:**

All source code must be documented with clear comments. At a minimum, every function/method must include:
- Description of purpose
- Expected Input parameters
- Expected Output / Return values
3. **README Document:**

The repository must include a clear `README.md` containing:
- **System Overview:** Brief description of the project.
- **Environment & Prerequisites:** Compiler/runtime versions, libraries, and external dependencies required to run the code.
- **Execution Guide:** Step-by-step instructions on how to build, run, and test the 10 functionalities (whether via command-line arguments, scripts, CLI, or GUI navigation steps).

General Information & Constraints

- You may use any programming language, framework, or IDE. However, remember that the evaluation takes place on an independent machine. Your project **must include clear environment setup instructions** or configuration files (e.g., `requirements.txt`, `package.json`, `pom.xml`, or Dockerfile) so it can be built and run without version mismatch issues.

AI Use and Interaction Record

Session: 20-21 October 2026, Europe/Berlin. This is a selected, normalized interaction record with main prompts, key responses and observed revisions. It is not a full archival export. Administrative quotations are preserved in MAIN_PROMPT.md; later discussions are summarized where indicated. Private credentials are not reproduced.

Models and Roles

Development assistant (initial Lab 3 Tasks A-C and assistant handoff integration): Codex desktop application, student-confirmed selector label **GPT-6.1 Sol**, used for Lab 3 Tasks A, B and C: elicitation/comparison, complete code generation and revision, tests, documentation and repository work. A separate backend build identifier was not exposed. The exact model used in earlier Lab 1-2 tutoring was not recorded.

Development assistant (subsequent local F11 evaluation/refinement and F06 recommendation update): **Gemini 4 Argon Medium** through **Google Antigravity**, used after opening the local repository in the IDE to inspect the uncommitted handoff state, build and run the multi-turn assistant evaluation (`tools/check_assistant_heldout_baseline.py`), refine `smartbrew/assistant.py` context handling and schema field order, specify and implement the expanded deterministic F06 recommendation scoring (`smartbrew/service.py`, `web/app.js`, `web/styles.css`), and expand the unit and browser verification suites to **33 unit tests** and **58 browser checks**. (An earlier Gemini web chat used by the student to draft initial assistant ideas had no direct repository or test access, and its model version was not recorded.)

Runtime drink assistant: local Ollama 0.40.6 with Qwen3 4B (`qwen3:4b`), added after the student approved the scope. This model interprets customer drink requests; it did not generate the coursework. The tested digest, outputs and timings are recorded separately in `evidence/current/local-ai-results.json`, `evidence/current/assistant-conversations.json`, `evidence/current/assistant-long-chat.json` and `evidence/current/assistant-heldout-postchange.json`.

Main Request

The supplied Lab 2 feedback, actual submission ZIP, Lecture 2/3 PDFs, system description and Lab 3 instructions form the main prompt in MAIN_PROMPT.md. The student asked to review Lab 2 against feedback and then complete Lab 3's elicitation, comparison, ten-function prototype and repository documentation.

Prompt strategy: ground the artifacts in supplied sources, use the lecture's requirement/backlog fields, preserve the submitted baseline, separate source mandates from proposed design choices, implement bounded workflows, and verify failures and ownership as well as successful demonstrations. This was an evolving conversation, not a fabricated sequence of separate experiments.

Assistant Responses and Student Clarifications

Selected excerpts below retain the original wording; execution observations are labelled separately.

Assistant:

> I'll read the feedback, submitted files, and lecture templates first. Then I'll use them to correct Lab 2 and build Lab 3's requirements, comparison, prototype, and documentation.

Assistant after reviewing feedback:

> The feedback gives Lab 2 a positive grade, but asks for fuller requirement descriptions, actor placement changes, and customization as the customer's starting point. The submitted PDF is missing the required preconditions and postconditions, so my earlier reassurance was too broad.

Student supplied the GitHub repository and confirmed the model label:

> GPT-6.1 Sol

The repository URL was https://github.com/kaantanidir/Software_Engi_1_LAB3.git .

Assistant implementation scope:

> For Lab 3, I'll build a local browser app around ten demonstrable features: accounts, menu browsing, customization, saved recipes, availability, recommendations, checkout/history, loyalty, receipts, and maintenance. Hardware inputs and external payments/email will be clearly labeled simulators; camera inference and a real local LLM will remain deferred.

The last clause describes the initial scope. The student later approved a real optional local drink assistant; camera inference remains deferred.

Lab 1 and Course-Policy Review

Student supplied the Lab 1 submission/feedback and general course AI restrictions. Prompt excerpt:

> I have provided all lab 1,2,3 instructions, feedback of 1 and 2, submission of 1 and 2.

Source-context summary: Labs 1-2 limit AI to explanations/grammar and prohibit generated submission text/models; Lab 3 permits end-to-end generation. The instructor recommends an additional experiment and asks for a personally written reflection if performed.

Assistant:

> The general instructions add an important distinction: Labs 1-2 prohibit submitting AI-generated text or models, while Lab 3 explicitly allows generation. I'll keep the generated Lab 2 correction labeled as a review aid and check that the repository makes that clear.

Student:

> do we need to, wouldnt disabling or mutating would decrease functionallityt of the project

Assistant:

> The message presents the extra experiment as recommended, rather than a required fourth task.
>
> A mutation would run only in a disposable test copy: deliberately break one rule, check whether a test catches it, then discard the copy. Your main project keeps all ten features.

No optional student experiment has been selected or claimed. The source baselines are preserved unchanged. The original submitted Lab 2 was AI-assisted in this conversation, so the report compares against the actual submitted group version and discloses that it is not an independently human-elicited baseline. Instructor clarification may be needed for that distinction.

Customer Review and Approved Revision

Discussion summary: the student reported that a newly created account returned to Alex on later login and showed Alex's records. They questioned whether the prototype worked beyond its prepared demonstration. The assistant inspected the code without making changes while the student requested discussion first.

Assistant:

> I found a concrete bug that explains this: the ordinary sign-in form also sends the hidden demo NFC value. The server prioritizes that value, so it signs you in as Alex instead of checking the email and password you entered.
>
> My browser tests missed it because they signed in as Alex, which looked successful either way.

Discussion summary: the student preferred a helpful local drink assistant over camera mood detection. Password change/recovery, local versus real email, Celery/Redis and Docker were discussed; password recovery and the infrastructure additions were left outside scope. The agreed plan also separated technician test controls from customer checkout, introduced an explicit demo-login action, completed order review/receipts and broadened browser tests.

Student approval:

> just try to make documantation a little bit better with ai part, also maybe check the general file organization as well. other than that, ok

Assistant:

> I'll proceed with the agreed plan, including a file-organization review. I'll make the documentation more readable and less repetitive while keeping the AI-use record accurate.

The documentation was reorganized around setup, architecture, testing and coursework records, with detailed provenance kept here and historical Lab 2 review aids separated from current Lab 3 documents. AI involvement was not relabeled as human authorship.

Generated Outputs and Observed Corrections

The application was generated end to end, not only reviewed. Its code responses are retained as source files and Git history:

- app.py and smartbrew/service.py: HTTP adapter, identity/ownership, saved recipes, ranking, purchases, receipts and maintenance.
- smartbrew/assistant.py and web/assistant.js: optional real local-model interpretation and confirmed recipe application.
- web/: responsive customer, technician and receipt views.
- tests/ and tools/: business-rule, model-validation, browser and live-inference checks; source audits and document generation.

Observed execution history, not invented prompts or student-written reflections:

1. Source review found missing requirement fields and wrong/empty Astah use-case descriptions that earlier name-only checks had missed.
2. Pending-wallet retry originally lost selected loyalty points; persistence and a retry regression were added.
3. Recommendation failure originally left the menu blank. A failing browser regression led to independent essential rendering and a bounded optional-ranking wait.
4. The student found the password/NFC mix-up. A new failing regression reproduced it; authentication now rejects mixed methods, normal fields start empty, and two-customer browser workflows verify actual identity and ownership.
5. Payment outcomes were moved from customer-submitted data to a protected, persistent technician setting. Requests attempting to override outcomes are rejected.
6. Personalized ranking was adjusted to quote the customer's real preference settings; selecting a recommendation applies those settings.
7. Real Qwen3 testing exposed an invented recipe for an unknown "usual" drink. Clarification rules/examples were refined and the failed response was retained. A cold start also exceeded the original timeout; the bounded wait and setup documentation were adjusted.
8. Final measurements, screenshots, model replies and remaining limitations are recorded in QA_RESULTS.md and evidence/. Passing selected tests is not proof that every possible model output or hardware integration is correct.

Student Review

Subsequent Gemini Proposal Integration

Follow-up user observation: generic repeated coffee descriptions for similar bitter/sweet requests.
Codex added a nine-turn live reproduction and explicitly separated recipe-following checks from
prose-quality observations. Explicit changes worked in this scenario; generic wording was still
observed. No frozen-conversation fix is claimed. See the implementation update and long-chat evidence.

The student used a separate Gemini chat to draft assistant context-retention/prompt/test changes and relayed its outputs here. The exact Gemini model label for that earlier web chat is unconfirmed. After the student asked Codex to continue and then "so?", Codex integrated the proposal locally, corrected test integration and ran real inference/browser checks. The successive failures, corrections, current status and attribution are recorded in `docs/development/ASSISTANT_IMPLEMENTATION_UPDATE.md`; the new live evidence is `evidence/current/assistant-conversations.json`. This contribution is not relabeled as student-authored work, and neither assistant's assurances substitute for execution evidence. Saved Gemini instructions are not proof of source-file access or test execution.

Subsequent Google Antigravity Session (Gemini 4 Argon Medium)

In a follow-up Google Antigravity IDE session using **Gemini 4 Argon Medium** with direct access to the local repository, the assistant first inspected the codebase and uncommitted changes read-only, then created a multi-turn evaluation script (`tools/check_assistant_heldout_baseline.py`) using isolated temporary databases.

1. **F11 Local Drink Assistant Evaluation and Fix:**
- The pre-change baseline (`evidence/history/assistant-heldout-baseline.json`, preserved unchanged) passed 5/8 turns and exposed four concrete issues: a leaked synthetic assistant placeholder message on open-ended follow-up (`A3`), failure to switch base drink from `latte` to `mocha`/`cocoa` when asked to make a drink chocolatey (`B2`), silent substitution of a hot vanilla latte for an unsupported iced caramel latte (`C1`), and unrequested extra espresso (`extra: 1`) when asked for sweetness beyond the 4-shot syrup cap (`C2`).
- Removing the synthetic assistant turn and replacing it with a structured trailing `system` context block (`current_proposal`), ordering `config` before `message` in `RESPONSE_SCHEMA`, and adding explicit rules against silent substitution and

unrequested extra espresso resolved `B2`, `C1`, and `C2` on the first post-change run (`evidence/history/assistant-heldout-postchange-first-run.json`, 7/8 passed, preserved unchanged) and all 8/8 turns after clarifying that alternative requests (`"what else"`) should return `kind="recipe"` (`evidence/current/assistant-heldout-postchange.json`, 8/8 passed). Both existing conversation scripts (`tools/check_assistant_conversations.py`, 12/12 turns, and `tools/check_assistant_long_chat.py`, 9/9 recipe turns) also passed.

2. **F06 Recommendation Specification Review and Implementation:**
- An initial F06 scoring draft contained arithmetic/sorting mismatches in its worked example and described per-drink recipe defaults not present in `Store.config()`. Before any code was written, the student required a corrected specification separating generic recipe defaults from drink-affinity lookup tables, removing caffeine/sleep claims (`"Late-hour comfort pick"`), renaming the count-based order reason from `"A recent favorite"` to `"A frequent choice"` (`+12` per completed order, capped at `+36`), defaulting omitted weather conditions to `"unknown"` (`0` points, no invented condition), adding `period_mode` (`"manual"` technician override vs. `"auto"` injectable local clock across `morning`, `afternoon`, `evening`, `night`) alongside `weather` (`cold`, `mild`, `warm`, `hot`) and `condition` (`unknown`, `sunny`, `cloudy`, `rainy`, `snowy`), and verifying customer ownership isolation.
- After design approval, `smartbrew/service.py`, `web/app.js`, `web/styles.css`, `tests/test_service.py`, and `tools/browser_check.cjs` were updated and verified against isolated temporary databases without touching `data/smartbrew.sqlite3`. All **33 unit tests** (`evidence/current/unit-tests.txt`) and **58 browser checks** (`evidence/current/browser-results.json`, with the initial post-logout quote-wait failure preserved separately in `evidence/history/browser-f06-first-run-failure.json`) passed.

Scope of Technical Record

This technical interaction record documents the design assumptions, baseline provenance, observed failures, automated test evidence and model limitations across the development and runtime models. It does not claim completion of the assigned reading or include a generated first-person student reflection.